SUPERVISED DRAFTERS

Nishchay Jasuja (njasuja) Alistair Cheong (acheong) Neil Purohit (neilpuro)

https://github.com/jasujanish/speculative_decoding_speedup

ABSTRACT

Tree-based speculative decoding accelerates large language model (LLM) inference by having a lightweight drafter model propose a tree of future tokens that the target model verifies in parallel. The throughput gain of this technique is sub-optimal when tree depth and verification budget (the number of nodes selected from the tree for verification) is fixed. Thus, Learning to Draft (LTD) employs Proximal Policy Optimization (PPO) to train a depth policy that determines when to halt tree expansion, and a size policy that selects how many nodes to verify. However, PPO is sample-inefficient, rendering LTD's training loop costly, while the size policy yields only marginal improvements. To create a dynamic drafter tree controller in a more efficient way, we replace LTD's PPO-trained depth policy with a supervised regressor trained from a dataset of collected throughput measurements. We also remove LTD's size policy, eliminating the need for LTD's alternating depth/size co-optimization. Our strongest variant, Titan, predicts whether any deeper stopping point improves throughput. Across Qwen3-8B and Qwen3-14B with EAGLE-3 drafters, Titan matches LTD's speedup while reducing training time by $\sim$87%, drafter calls by $\sim$95%, and target calls by $\sim$80% relative to co-optimized LTD. This indicates that supervised learning can be an efficient alternative to PPO for training a draft tree controller.

1 INTRODUCTION

Large language model (LLM) serving is increasingly constrained by inference cost. Autoregressive decoding emits one token per forward pass of the target model, causing latency to scale linearly with output length. Speculative decoding alleviates this bottleneck by pairing a large target model with a cheaper draft model. The draft model proposes future tokens and the target model verifies multiple candidates in a single parallel pass (Chen et al., 2023). When several drafted tokens are accepted, a single forward pass of the target model advances generation by multiple tokens. Thus, speculative decoding provides a highly promising initial idea for scaling LLM inference, which is crucial as LLMs continue to diffuse across various fields.

Tree-based speculative decoding extends this idea by having the draft model generate a tree of candidate continuations rather than a single sequence (Miao et al., 2023). Once the tree is constructed, the target model verifies the top V nodes ranked by cumulative path score in one forward pass with a tree-attention mask. A deeper tree or larger verification budget can yield a longer accepted path, but may also waste drafter compute on low-quality branches. Thus, the central control problem is therefore how to set the tree depth D and verification budget V for the current decoding state.

Learning to Draft (LTD) addresses this control problem by introducing a draft tree controller that dynamically sets D and V for the current decoding state. It adds two lightweight multilayer perceptrons (MLPs) trained with Proximal Policy Optimization (PPO) on top of the EAGLE-3 drafter: a depth policy that decides when to halt tree expansion, and a size policy that selects V once the tree is constructed (Zhang et al., 2026; Schulman et al., 2017).

While LTD improves throughput substantially, it incurs a high initial training cost. PPO is an on-policy algorithm, so each update step requires freshly collected roll-outs. Generating these roll-outs involves repeated calls to both the drafter and target models, which dominates the training budget. Across hundreds of thousands of PPO steps over training, these calls incur a significant computational overhead. In our reproduction, this 6-stage training loop (with 100,000 environment interactions for each phase) took ~ 9 hours on Qwen3-8B and ~ 11 hours on Qwen3-14B (Modal 1x H100 80GB). Moreover, LTD's own ablations show that its size policy contributes only marginal gains, yet nearly half the training budget is spent on co-optimizing it.

These observations motivate a more efficient approach to constructing such a controller. We observe that LTD's depth policy tries to find the optimal stop/continue decision with reinforcement learning (RL), but it can already be obtained directly from the training data by sweeping over various tree depths, measuring the throughput at each depth, and calculating if further expansion would improve throughput beyond the current stopping point. Thus, instead of rely-

ing on RL, we train a supervised regressor to predict this throughput gain directly. This formulation enables data reuse as a single sweep yields multiple labeled examples that can be used for multiple gradient updates.

We investigate two variants of this supervised depth controller, Base and Titan (Section 5). Base predicts whether the next depth yields higher throughput than the current depth (a greedy objective), whereas Titan predicts whether any deeper stopping point would outperform the current one. Titan additionally has a larger parameter count and slightly richer input features. Following LTD, we train on the HumanEval dataset (Chen et al., 2021) and evaluate on MT-Bench (Zheng et al., 2023) and GSM8K (Cobbe et al., 2021). Of the two variants, Titan proves strongest. It matches LTD's speedup while reducing training time by $\sim$87%, drafter calls by $\sim$95%, and target calls by $\sim$80% relative to co-optimized LTD (Section 7). This indicates that supervised learning can serve as an efficient alternative to PPO for training a draft tree controller.

We make three main contributions.

1. We formulate the training of a draft tree controller as a supervised learning problem rather than an reinforcement learning problem.
2. We implement a data collection pipeline that sweeps across draft tree depths and records the throughput obtained by stopping at each depth.
3. We train two types of supervised depth controllers and compare them against vanilla autoregressive decoding, static EAGLE-3, non-co-optimized LTD, and co-optimized LTD. Our strongest variant, Titan, matches LTD's speedup while substantially reducing training time and model-call cost.

2 Problem

2.1 Dynamic Draft Tree Controller for Tree-based Speculative Decoding

In tree-based speculative decoding, the drafter uses the current prefix to construct a depth-D tree of candidate continuations. Each node represents one token and has a path score given by the product of the conditional draft probabilities along the path to that node. After the tree is constructed, the system selects the top V nodes by this path score for verification by the target model. V is therefore called the verification budget.

These quantities are difficult to choose statically. A fixed pair (D, V) applies the same amount of draft-tree expansion and verification to every decoding context, which can be suboptimal as different contexts may require different draft trees. When the task is easy and the drafter is likely to agree with the target model, a deeper tree can help the system advance many tokens in one verification pass. When the task is harder and the drafter is less reliable, expanding too deeply wastes drafter compute on branches that the target model later rejects. It is therefore preferable to set D and V dynamically based on the current decoding context.

This paper colloquially refers to the above problem as the dynamic draft tree controller problem. Given the current decoding context and drafter state, the controller must decide how to set D and V to control how the draft tree is generated and verified. When the controller is learned, the corresponding training problem is how to obtain a policy for choosing these quantities efficiently.

2.2 Research Questions

The project focuses on the following research questions.

1. **Is reinforcement learning necessary for training the depth policy?** LTD trains its depth policy with PPO, but the depth decision can also be labeled using measured throughput at different stopping depths. We test whether these supervised labels are sufficient to train a competitive depth controller.
2. **Which supervised target is better for training the regressor?** We compare a greedy next-depth target against a best-future target that asks whether any deeper stopping point beats stopping now.

3 Related Work

3.1 Dynamic Draft Trees

SpecInfer introduced tree-based speculative decoding, establishing the structure and size of the tree as key determinants of speedup (Miao et al., 2023). Separately, EAGLE improved the drafter by using feature-level prediction: instead of predicting tokens directly, EAGLE predicts the hidden representations of the target model's penultimate layer and reuses the target model's LM head to convert them into token probabilities (Li et al., 2024a). EAGLE-2 pairs EAGLE-style drafting with dynamic tree construction, using cumulative drafter confidence along each path to decide which candidates to expand further (Li et al., 2024b). EAGLE-3 further improves the drafter by predicting a fused representation that combines low-level, middle-level, and high-level target features (Li et al., 2025).

Both LTD and our experiments build on the EAGLE-3 drafter. EAGLE-3, like EAGLE-2, uses fixed runtime budgets for maximum depth, expansion width, and verification size during tree construction. The problem of introducing budgets is left to the runtime controller. As such, our scope is limited to training the lightweight MLP that controls

the draft tree. We do not modify the target model or the EAGLE-3 drafter.

3.2 Training-Free Heuristics

A second line of work argues that fixed budgets are suboptimal and proposes hand-designed rules to adapt them. OPT-Tree constructs a new tree at each decoding step under a fixed node budget, scoring candidate nodes by the product of draft probabilities along the path and keeping the subtree with the largest expected acceptance length (Wang et al., 2024). DySpec also builds the tree at runtime, maintaining a frontier of expandable nodes and greedily expanding the most promising node until the token budget is exhausted (Xiong et al., 2024). Dynamic Depth Decoding focuses specifically on EAGLE-2's beam-search depth. It keeps the beam width fixed and stops further draft-model calls once the total confidence of the current beam falls below a threshold (Brown et al., 2024). Further, TALON spends a fixed node budget as either deep-and-narrow growth for confident contexts or shallow-and-wide growth for uncertain contexts (Liu et al., 2026). These methods show that context-dependent tree budget allocation is valuable. However, these methods primarily use training-free analytical rules or cost heuristics, so accuracy hinges on hand-tuned thresholds and simplified cost assumptions. As such, methods that incorporate a learned controller designed to directly optimize throughput, like ours, have been shown to be more effective than these methods (Zhang et al., 2026).

3.3 Learned Draft Controllers

The closest methods to this paper train or adapt a policy for speculative decoding decisions rather than relying only on hand-written thresholds. AdaEAGLE trains a lightweight MLP to predict how many sequential draft tokens to generate, but it is limited to linear drafting and does not control tree depth or width. (Zhang et al., 2024). As previously mentioned, LTD improves upon AdaEAGLE by formulating adaptive tree control as a reinforcement learning problem and training two separate policies, a size policy that builds off the AdaEAGLE objective and a depth policy that determines the depth of the generated tree (Zhang et al., 2026). Our work improves on these learned controller approaches in three key areas. First, we replace LTD's sample-inefficient reinforcement learning loop with supervised learning, allowing collected depth-sweep data to be reused across training epochs. Second, unlike AdaEAGLE, our trained controllers directly operate on the structure of the drafter three. Third, we remove the size-control objective used by AdaEAGLE and LTD and focus on tree-depth control. This gives us a more expressive controller than AdaEAGLE, which only predicts sequential draft length, while keeping the training objective simpler than LTD's two-policy depth/size co-optimization.

4 System Overview

Figure 1 summarizes the full pipeline. Our method first constructs a supervised depth-control dataset from HumanEval prompts, then trains a lightweight regressor that is inserted into EAGLE-3 tree growth. At inference time, the regressor is called repeatedly while the draft tree is being expanded. Its only responsibility is to decide whether expanding one more depth level is likely to improve throughput. Following the original LTD paper, we fix the target model and EAGLE-3 drafter and use verification budget $V = 60$ to ensure a fair comparison.

5 Method

5.1 Supervised Depth Regressors

We train a supervised regressor to replace LTD's PPO-trained depth policy. The controller is implemented as a small PyTorch MLP and is invoked during draft-tree construction. At depth d, it receives features describing the current prefix and the partially constructed draft tree, and outputs a scalar predicted delta $\widehat{\Delta}(d)$. This value is interpreted as the predicted throughput gain from continuing to expand the tree rather than stopping at the current depth. The inference rule is therefore

$$\text{continue expanding} \iff \widehat{\Delta}(d) > 0. \tag{1}$$

If the predicted delta is non-positive, tree expansion stops and the target model verifies the current tree. If the predicted delta is positive, the drafter expands one more layer, up to a maximum depth of 12. This controller only controls depth. Unlike LTD, it does not choose the verification size; we fix $V = 60$ nodes for all supervised runs.

We evaluate two supervised regressors, summarized in Table 1. Base is the simplest version: it asks whether the next depth is better than the current depth. Titan uses a larger MLP and a best-future target: it asks whether any deeper stopping point is better than stopping now. We train both models to compare the effectiveness of the greedy and optimal objectives and identify whether our overhead policies can effectively learns both objectives.

5.2 Dataset Construction

The supervised dataset is built from the same HumanEval-style prompt subset used in the LTD experiments. This subset contains 80 coding prompts. We split it deterministically with seed 42 into 64 training prompts and 16 held-out validation prompts. The 64 training prompts are used to collect supervised rows. The 16 held-out prompts are not used for row collection; they are used later to select the final checkpoint by actual decoding speedup.

Dataset collection proceeds until the number of supervised

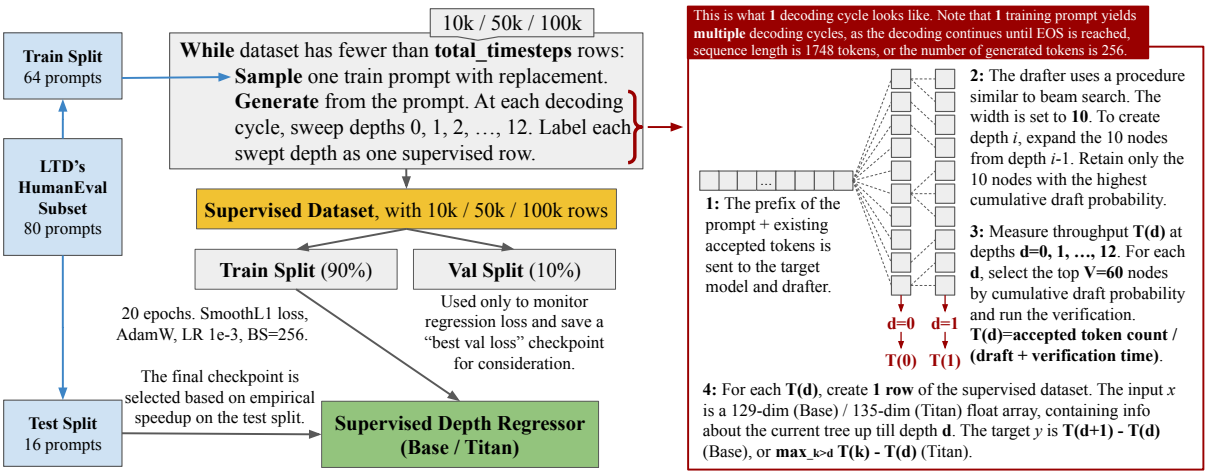

Figure 1. Overview of the supervised depth control pipeline. Our supervised regressor is created from HumanEval data, and later evaluated using MT-Bench and GSM8K data.

Table 1. Supervised depth-regressor variants. Both models output a predicted throughput delta and continue tree expansion when that delta is positive.

Regressor	Input dims	Hidden architecture	Params	Output intuition
Base	129	1024	134K	Predicted gain from expanding exactly one more depth.
Titan	135	$1664 \to 384 \to 128$	915K	Predicted gain from reaching the best deeper stopping point.

rows reaches the requested budget, which is 10K, 50K, or 100K in our experiments. At the start of each generation episode, we sample one of the 64 training prompts with replacement and begin decoding from that prompt. A single sampled prompt can produce many speculative decoding cycles. After each cycle, the accepted tokens are appended to the prefix, and if generation has not ended, the next cycle starts from this updated prefix. The episode ends when the model emits an end token, reaches the sequence-length limit of 1748 tokens, or generates 256 new tokens.

Within each speculative decoding cycle, we sweep all possible stopping depths from $d = 0$ to $d = 12$. We set the maximum depth to 12, the same as LTD, for direct comparability. The sweep starts from the current prefix, which consists of the original prompt plus any tokens already accepted in earlier cycles. The EAGLE-3 drafter then constructs the draft tree using a beam-search-like expansion rule with frontier width 10, as was done in the LTD paper. At the first draft step, the drafter produces token scores and retains the top 10 frontier nodes. To expand one more depth level, the drafter expands the current 10 frontier nodes; each frontier node contributes its top 10 children, giving 100 candidate children. These children are scored by cumulative draft probability along the path from the root. Equivalently, in log-probability space, their log scores are summed along the path. The top 10 children by this cumulative score become the next frontier. Nodes that are not retained can still remain in the already constructed tree, but they will not be expanded further.

One subtle point is that depth $d = 0$ is not a state with no drafter output. It is the first stopping point after the initial EAGLE proposal layer has been constructed. Thus $T(0)$ measures the throughput of stopping immediately after the shallow initial tree is available, before any additional beam-style expansion. Each larger d measures the throughput of stopping after more expansion layers have been added.

5.3 Throughput Measurements and Labels

For each depth d, we measure the throughput that would be obtained by stopping at that depth. First, the current tree is finalized by selecting the top $V = 60$ nodes according to cumulative draft path score. The target model then verifies those selected nodes in a single tree-attention forward pass. Verification returns the accepted path length for that speculative cycle. Let $a(d)$ be the number of accepted tokens, $t_{\text{draft}}(d)$ be the drafter time accumulated up to depth d, and $t_{\text{verify}}(d)$ be the target-model verification time at depth d. We define

$$T(d) = \frac{a(d)}{t_{\text{draft}}(d) + t_{\text{verify}}(d)}. \tag{2}$$

This produces 13 measured values for one speculative cycle:

$$T(0), T(1), \ldots, T(12). \tag{3}$$

Each measurement $T(d)$ becomes one supervised row. The input x_d is a float vector describing the current tree state at depth d. For Base, this vector has 129 dimensions: the first 100 dimensions store cumulative draft-frontier scores, the next 14 dimensions repeat the normalized context length, the next 14 dimensions repeat the normalized draft depth, and the final dimension stores the entropy of the current frontier scores. The first 128 dimensions are inherited from LTD's depth-policy observation, while the entropy dimension is added for our supervised controller. Titan uses the same 129 dimensions plus six additional features: maximum frontier score, mean frontier score, score standard deviation, minimum frontier score, frontier entropy change from the previous depth, and the drafter's log probability for generating an end token at the current frontier. We allocate greater representational capacity and a richer feature vector to the Titan model as it attempts to learn a more complex relationship than the Base model.

Base uses a greedy next-depth target:

$$y_d^{\text{Base}} = T(d+1) - T(d). \tag{4}$$

Titan instead uses a best-future target:

$$y_d^{\text{Titan}} = \max_{k>d} T(k) - T(d). \tag{5}$$

At the final depth, this target is set to zero because no deeper depth is available. The Titan label better matches the semantics of a stopping decision: the controller should continue if there exists a deeper stopping point that is better than stopping now, even if the immediate next depth is not itself the best point.

5.4 Training and Checkpoint Selection

The collected supervised rows are split 90/10 into a training split and an internal regression-validation split. The model is trained for 20 epochs with SmoothL1 loss, AdamW, learning rate 10^{-3}, and batch size 256. We run all training runs on one H100-80GB GPU on Modal.

The final supervised depth regressor is not selected by regression loss alone. Candidate checkpoints are evaluated on the 16 held-out HumanEval prompts, and we promote the checkpoint with the highest measured decoding speedup. This matters because a low regression loss on throughput deltas does not always imply the best end-to-end stopping behavior. The held-out HumanEval split therefore plays the role of checkpoint selection, while MT-Bench and GSM8K are reserved for the main benchmark evaluation.

5.5 Why Supervision Can Be Cheaper Than PPO

The supervised formulation is designed to reduce training cost in three ways. First, it reuses labels. A single depth sweep produces labels for all 13 stopping depths in the same speculative cycle, and each labeled state can be revisited across multiple epochs of supervised training. PPO does not have this property: it collects on-policy rollouts and uses them for a limited number of policy-gradient updates before new rollouts are required. Second, the supervised formulation removes the need for depth/size co-optimization. LTD trains both a depth policy and a size policy, then alternates between them so that the two policies adapt to each other. Our method fixes $V = 60$ and trains only the depth controller. This deliberately removes some adaptivity, but it also eliminates the alternating curriculum that accounts for a large part of LTD's training time. Third, the optimization problem is simpler. The supervised regressor directly predicts measured throughput deltas. It does not require sparse terminal rewards, advantage estimation, policy-ratio clipping, entropy tuning, or other PPO-specific stability choices. This simplicity is the reason we expected a small MLP trained on measured depth sweeps to be competitive with an RL policy while requiring far fewer model calls.

5.6 Libraries and Language

We implemented the supervised depth-control pipeline in Python using PyTorch. We adapted the open source EAGLE-3 and LTD code bases alongside Qwen3 family models. Our additions were the sweep-based data collector, the Base and Titan PyTorch MLP policies, the supervised training loop, checkpoint selection, and the inference-time hook that calls the regressor during EAGLE-3 tree expansion.

6 Evaluation

6.1 Models and Data

We evaluate two target-model scales: Qwen3-8B and Qwen3-14B. Each target model is paired with its matching AngelSlim EAGLE-3 drafter. The supervised controller is trained from the 64-prompt HumanEval training split described above, with checkpoint selection on the 16-prompt HumanEval validation split. Final benchmark evaluation uses MT-Bench and GSM8K, each containing 80 prompts in our evaluation suite (Zheng et al., 2023; Cobbe et al., 2021). All experiments are run on one H100-80GB GPU.

6.2 Baselines

We compare against four inference settings:

- **Vanilla AR**: target-model autoregressive decoding without speculative drafting.

- **Static EAGLE-3**: EAGLE-3 speculative decoding with fixed depth and fixed verification budget.
- **LTD no-co-op**: the initial LTD depth and size policies before alternating co-optimization.
- **LTD co-op**: LTD after the alternating size/depth co-optimization schedule.

Conceptually, the baselines are vanilla decoding, static EAGLE-3, and LTD. We report LTD with and without co-optimization because we observed varying training costs and speedups between these variants in our runs.

The 10K, 50K, and 100K labels in our tables are training-budget labels rather than a shared count of examples. For our supervised method, the label denotes the requested minimum number of labeled depth states collected from depth sweeps. For LTD, it denotes the PPO environment-step budget assigned to each policy-training stage: following LTD, a depth-policy step is a draft/stop action, while a size-policy step is one verification-size decision for a speculative cycle. Thus, LTD no-co-op uses the stated budget for each initial policy stage, and LTD co-op uses the stated budget for each additional alternating co-optimization stage; for example, a 100K co-op run means 100K PPO steps per stage, not 100K total stored records.

6.3 Metrics

The primary inference metric is mean speedup over vanilla autoregressive decoding. For a method m, this is the mean tokens-per-second of m divided by the mean tokens-per-second of vanilla AR on the same benchmark subset. Thus, vanilla AR has speedup 1.00 by definition. We also report τ, the average number of accepted tokens per draft-and-verify cycle, when it is useful for interpretation.

The primary training-cost metrics are wall-clock training time, drafter forward calls, and target forward calls. For supervised methods, these costs include depth-sweep data collection, supervised training, and validation-based checkpoint selection. For LTD, they include PPO training and validation-based checkpoint selection. Main hyperparameters are fixed across supervised runs unless otherwise noted:

- Training budgets: 10K, 50K, and 100K.
- Epochs: 20.
- Maximum draft depth: 12.
- EAGLE top-k: 10.
- Fixed verification size: 60 nodes.
- Supervised batch size: 256.
- Supervised learning rate: 10^{-3}.

Table 2. Matched aggregate comparison against LTD. Values are mean percent changes over all six model/budget settings.

LTD ref.	Policy	Spd.	Time	Draft	Target
co-op	Base-20	-4.54	-85.26	-93.68	-80.17
no-co-op	Base-20	-4.04	-74.77	-89.99	-64.43
co-op	Titan-20	**+0.15**	**-87.16**	**-94.88**	**-80.36**
no-co-op	Titan-20	+0.67	-78.02	-92.06	-64.78

7 RESULTS

7.1 Overall Matched Comparison

For each supervised controller and each LTD reference, we compare the same six settings: two target-model sizes times three training budgets. No setting is dropped or reweighted after observing the results. This aggregate view answers the question most relevant to our project: whether a supervised depth controller can give LTD-level average speedup while requiring substantially less training.

For every metric M, we compute the percent change

$$\Delta_M = \frac{M_{\text{supervised}} - M_{\text{LTD}}}{M_{\text{LTD}}} \times 100. \tag{6}$$

Positive speedup indicates that the supervised methods has faster inference. Negative time and calls indicate that the supervised method has lower training cost.

Table 2 is the source of the headline claim. Relative to co-optimized LTD, Titan is essentially speed-neutral on average (+0.15%) while reducing wall-clock training time by 87.16%, drafter calls by 94.88%, and target calls by 80.36%. Base also reduces cost, but its average speedup is slightly lower. Our key finding that supervised controllers are a cost-efficient alternative to LTD is supported by our more detailed results below.

7.2 Per-Setting Speedup

Table 3 reports the raw mean benchmark speedups used in the aggregate comparison in greater detail. The row-level picture is mixed, which is why the aggregate in Table 2 should not be interpreted as Titan being uniformly better than LTD. The results in the table highlight that, in many settings, LTD does show higher performance than the Titan and Base models. Notably, Titan has a large gain on Qwen3-8B at the 10K budget, but it trails co-optimized LTD by smaller margins in several Qwen3-14B settings.

Base is less consistent. It achieves the single best Qwen3-8B 50K result at 2.7300x, but it underperforms on Qwen3-14B 10K and 50K. This supports the motivation for Titan's best-future target. Greedy next-depth labels are a simple target that can be learned by a much smaller network, but they introduce high variance as a single next layer can be

Table 3. Mean benchmark speedup over vanilla autoregressive decoding. Static is the fixed EAGLE-3 baseline; LTD no and LTD co denote LTD before and after co-optimization.

Model	Budget	Vanilla	Static	LTD no	LTD co	Base-20	Titan-20	Best
Qwen3-8B	10K	1.0000	2.1222	2.1191	1.9561	2.1507	**2.3455**	Titan-20
Qwen3-8B	50K	1.0000	2.0930	2.2742	2.5872	**2.7300**	2.4016	Base-20
Qwen3-8B	100K	1.0000	2.0772	**2.4003**	2.3654	2.3807	2.2665	LTD no
Qwen3-14B	10K	1.0000	2.1483	2.2570	**2.4645**	1.7086	2.3470	LTD co
Qwen3-14B	50K	1.0000	2.1831	2.4989	**2.5019**	2.1645	2.4631	LTD co
Qwen3-14B	100K	1.0000	2.2559	**2.7346**	2.5152	2.5350	2.4811	LTD no

Table 4. Training cost comparison for the same 20-epoch supervised settings used in Table 3. Time is wall-clock hours; drafter calls and target calls count model forward calls consumed while producing the final controller.

Model	Budget	Method	Speedup	Time (h)	Drafter (M)	Target (K)
Qwen3-8B	10K	LTD no-co-op	2.1191	0.71	0.32	45.7
Qwen3-8B	10K	LTD co-op	1.9561	1.22	0.59	78.0
Qwen3-8B	10K	Base-20	2.1507	0.24	0.07	24.6
Qwen3-8B	10K	Titan-20	2.3455	0.20	0.04	24.6
Qwen3-8B	50K	LTD no-co-op	2.2742	3.14	1.58	200.9
Qwen3-8B	50K	LTD co-op	2.5872	5.41	2.48	363.0
Qwen3-8B	50K	Base-20	2.7300	0.57	0.07	61.1
Qwen3-8B	50K	Titan-20	2.4016	0.48	0.08	60.9
Qwen3-8B	100K	LTD no-co-op	2.4003	5.13	3.12	383.8
Qwen3-8B	100K	LTD co-op	2.3654	8.88	4.63	714.7
Qwen3-8B	100K	Base-20	2.3807	1.15	0.14	114.1
Qwen3-8B	100K	Titan-20	2.2665	1.28	0.13	114.8
Qwen3-14B	10K	LTD no-co-op	2.2570	0.82	0.30	45.2
Qwen3-14B	10K	LTD co-op	2.4645	1.35	0.44	79.4
Qwen3-14B	10K	Base-20	1.7086	0.27	0.06	17.3
Qwen3-14B	10K	Titan-20	2.3470	0.23	0.06	16.5
Qwen3-14B	50K	LTD no-co-op	2.4989	3.18	1.54	197.2
Qwen3-14B	50K	LTD co-op	2.5019	5.33	2.31	364.7
Qwen3-14B	50K	Base-20	2.1645	0.71	0.09	61.3
Qwen3-14B	50K	Titan-20	2.4631	0.59	0.07	60.5
Qwen3-14B	100K	LTD no-co-op	2.7346	6.15	3.03	379.6
Qwen3-14B	100K	LTD co-op	2.5152	11.05	4.47	709.9
Qwen3-14B	100K	Base-20	2.5350	1.35	0.12	114.4
Qwen3-14B	100K	Titan-20	2.4811	1.02	0.12	114.4

unhelpful even though a deeper tree would eventually be worth reaching.

7.3 Training Cost

Table 4 reports the training cost required to produce each learned controller. The speedup column is repeated from Table 3 so that accuracy and cost can be read together.

The cost savings are large and consistent. At the 100K budget on Qwen3-14B, Titan trains and validates in 1.02 hours versus 11.05 hours for co-optimized LTD, while reaching 2.4811x speedup versus 2.5152x for co-optimized LTD. At the 50K budget on Qwen3-14B, Titan reaches 2.4631x versus 2.5019x for co-optimized LTD, while reducing wall-clock training time from 5.33 hours to 0.59 hours. Thus, replacing PPO (an on-policy reinforcement learning method) with supervised learning dramatically reduces the training cost of our speculative decoding controller.

8 Discussion

The results suggest that the depth decision in tree-based speculative decoding has enough directly measurable signal for supervised training. The supervised controller does not need to infer the reward structure from sparse on-policy feedback. Instead, it observes measured throughput curves over draft depth. This gives a relatively dense learning signal: every speculative cycle yields one label per depth, and those labels can be reused over training epochs.

The comparison also clarifies the role of co-optimization. LTD's co-optimized policy is not uniformly better than its non-co-optimized policy in our Qwen3 runs; for example, Qwen3-8B at 100K and Qwen3-14B at 100K both favor the non-co-optimized LTD row in Table 3. This does not imply that size/depth co-optimization is useless in general. It does suggest that, for these model and drafter pairs, most of the benefit can be captured by a depth controller with a

fixed verification size. Avoiding the size policy is therefore a reasonable simplification when training cost is a first-order concern.

Titan appears to be the safer supervised objective. Base sometimes wins, especially on Qwen3-8B at the 50K budget, but it is more volatile. Its target asks whether the next layer alone improves throughput, which can be a poor proxy for the stopping decision if useful gains occur only after more than one additional expansion. Titan instead asks whether any deeper stopping point is better, which better matches the action taken by the controller: continue now if continuing can eventually lead to a better stopping depth.

9 Limitations and Future Work

Our current experiments are limited to a small set of models (Qwen3-8B and Qwen3-14B with AngelSlim EAGLE-3 drafters) evaluated on one hardware setup (Modal 1x H100 80GB). Further ablations across model families, drafter architectures, and hardware platforms would provide greater insight into determining how effectively the Titan and LTD controllers perform across varying conditions.

Finally, like LTD, our policies focus on controlling tree depth, but tree width remains fixed. Dynamically controlling both tree depth and tree width would allow the system to adapt its draft budget more completely, using width when the drafter is uncertain and depth when it is confident. Extending our supervised labeling approach to this joint decision is a natural next step.

10 Conclusion

We presented Supervised Drafters, a supervised alternative to LTD's PPO-trained depth policy for tree-based speculative decoding. We reformulate the task of dynamically controlling drafter tree depth from a reinforcement learning problem to a supervised learning problem. By sweeping draft depths and labeling each state with its measured throughput, we reduce the stopping decision to direct regression, enabling full data reuse across epochs. Titan, our best-future supervised variant, matches LTD-level speedup while substantially reducing training time and model-call cost. These results suggest that throughput-labeled depth sweeps provide a practical, data-efficient substitute for PPO in draft-tree control, and suggest a path toward lightweight supervised controllers for joint depth and width decisions.

REFERENCES

Brown, O., Wang, Z., Do, A., Mathew, N., and Yu, C. Dynamic depth decoding: Faster speculative decoding for LLMs. *arXiv preprint arXiv:2409.00142*, 2024.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the International Conference on Machine Learning*, 2024a.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2024b.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.

Liu, T., Lv, Q., Shen, Y., Sun, X., and Sun, X. TALON: Confidence-aware speculative decoding with adaptive token trees. *arXiv preprint arXiv:2601.07353*, 2026.

Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., Zhu, A., Yang, L., Shi, X., et al. SpecInfer: Accelerating generative large language model serving with tree-based speculative inference and verification. *arXiv preprint arXiv:2305.09781*, 2023.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Wang, J., Su, Y., Li, J., Xia, Q., Ye, Z., Duan, X., Wang, Z., and Zhang, M. OPT-Tree: Speculative decoding with adaptive draft tree structure. *arXiv preprint arXiv:2406.17276*, 2024.

Xiong, Y., Zhang, R., Li, Y., Wu, T., and Zou, L. DySpec: Faster speculative decoding with dynamic token tree structure. *arXiv preprint arXiv:2410.11744*, 2024.

Zhang, J., Yu, Z., Wang, L., Yang, N., Yu, E. J., Li, Z., Song, Y., Zhu, D., Zhang, X., Wei, F., and Li, S. Learning to draft: Adaptive speculative decoding with reinforcement learning. *arXiv preprint arXiv:2603.01639*, 2026.

Zhang, S., Wang, H., Ma, D., Zhu, Z., Chen, L., Lan, K., and Yu, K. AdaEAGLE: Optimizing speculative decoding via explicit modeling of adaptive draft structures. *arXiv preprint arXiv:2412.18910*, 2024.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.